CyberTrace Integration, Architecture & End-to-End Delivery Report

Project: CyberTrace: DNS Exfiltration Detection Using Network Traffic Analysis
Repository: `vid6848/dns-exfiltration-detection`
Author: Soham Prajapati (GitHub: `@Soham-Prajapati`, Git Email: `soham.prajapati24@spit.ac.in`)
Assigned Area: Project Coordination & Integration (API/data contracts, repo structure, PR reviews, integration branch, end-to-end demo, setup guide and integrated build)
Evaluation Scope: Diagnostic Development Evaluation (Outer Fold 5) — Not an independent final test or production generalizability claim
Report Date: 2026-10-13
Status: Complete & Verified

---

1. Executive Summary & Assigned Role Scope

Per the approved **CyberTrace Team Work Allocation** (`CyberTrace_Team_Work_Allocation.pdf`), my primary ownership as Project Coordination & Integration Lead comprises:

> *"Own task board, API and data contracts, repository structure, pull request reviews, integration branch, and end to end demo. Deliver setup guide and integrated build."*

Prior to this integration phase, the project consisted of separately delivered component pull requests (#1 through #5) merged into `main`, but lacked an operational end-to-end integration layer. Crucial endpoints were absent, batch behavioral feature extraction suffered from state-destruction bugs, CORS was not configured, the repository setup instructions failed out-of-the-box, and dashboard telemetry was entirely disconnected from backend inference.

On the dedicated integration branch `integration/soham-e2e`, I resolved all identified regressions, formalized authoritative contracts in `docs/CONTRACTS.md`, unified project dependencies and automation in `requirements.txt` and `Makefile`, completed the full 5-endpoint REST API suite plus multipart CSV processing in `api/main.py`, connected the React dashboard to live telemetry feeds, conducted exhaustive reviews of PRs #1–#5, investigated the root cause of `source_hashes_match = false`, established CI/CD and repository governance, and authored a reproducible demo guide (`docs/DEMO.md`).

---

2. Baseline Status at Handoff & Evidence of Gaps

An exhaustive audit of `main` at commit `8321f8a` and engineering handoff documentation (`HANDOFF_TO_PRANAY_AND_SOHAM.pdf`) revealed several integration gaps:

2.1 Broken Environment Reproduction (`requirements.txt`)

- **Evidence:** Running `pip install -r requirements.txt` as instructed in both the project documentation and the initial draft report failed immediately with: `text ERROR: Could not open requirements file: [Errno 2]`

`No such file or directory: 'requirements.txt'`

- **Root Cause:** Dependencies were split across `requirements-data.txt`, `requirements-training.txt`, and `requirements-dev.txt`, with no top-level `requirements.txt` existing in the repository root.

2.2 Incomplete API Endpoint Implementation

- **Evidence:** In `api/main.py` (delivered in PR #4), only `GET /`, `GET /health`, `POST /predict`, and `POST /analyze` were defined.
- **Omissions:** The three required endpoints mandated in Handoff Section 4 were completely missing:
- `GET /alerts`: Required for live feed of suspicious activity.
- `GET /statistics`: Required for real aggregated traffic counts.
- `GET /model-performance`: Required to dynamically serve offline benchmark figures.
- `POST /analyze/csv`: Missing multipart CSV file upload capability.

2.3 Batch Behavioral Extractor Re-Instantiation Bug (Confirmed)

- **Evidence:** In PR #4 (`api/main.py:139-142`): `python for query in batch.queries: ... # Fresh behavioral context for this query. behavioral_extractor = BehavioralFeatureExtractor(window_seconds=60.0) behavioral_features = behavioral_extractor.extract(record) features.update(behavioral_features)`
- **Impact:** Re-instantiating the extractor inside the loop destroyed sliding-window history between queries. Every single query was treated as the first query (`time_interval = -1.0`, `query_frequency = 1.0`, `unique_subdomains = 1.0`), preventing multi-query burst detection. Furthermore, incoming queries were not sorted chronologically.

2.4 Missing CORS Middleware & Zero API Automated Tests

- **Evidence:** PR #4 contained zero CORS middleware, causing browser requests from the Vite dashboard on port 3000 to be blocked by browser CORS policy. PR #4 was merged without a single unit test (`tests/test_api.py` did not exist).

2.5 Unwired Frontend Placeholders

- **Evidence:** In `frontend/src/` (delivered in PR #5):
- `LiveStatsCards.jsx`: Rendered hardcoded static placeholder strings (`--`).
- `LiveAlertsTable.jsx`: Completely disconnected from `/predict` and `/analyze`; remained static even when attacks were flagged.
- `OfflineModelPerformance.jsx`: Loaded hardcoded static constants from a local file (`offlineEvaluationData.js`) rather than consuming the backend API.
- `package.json`: Lacked a `"test"` script, causing `npm test` to fail immediately.

---

3. Work Delivered on `integration/soham-e2e`

Every contribution was delivered through small, conventional commits adhering strictly to the project rules:

Commit Hash	Commit Message	Files Added / Modified	Description & Technical Rationale
`be83592`	`docs(reviews): add review of Param remediation branch and post-hoc reviews for PRs #1–#5`	`docs/reviews/param-remediation.md`, `PR-1.md` through `PR-5.md`, `README.md`	Formal technical code reviews for open remediation branch and all merged PRs.
`09eb495`	`Merge branch 'origin/fix/backend-frontend-remediation' into integration/soham-e2e`	`api/main.py`, `tests/test_api.py`, `frontend/package.json`, `client.test.js`	Merged approved remediation branch adding CORS, chronological batch sorting, and baseline API tests.
`168e82c`	`docs(contracts): add authoritative API and data contracts specification`	`docs/CONTRACTS.md`	Defined authoritative schemas, 10-feature manifest, 0.89 threshold, capture-isolation, and terminology policies.
`75bf8be`	`build: add unified requirements.txt and workflow Makefile`	`requirements.txt`, `Makefile`	Resolved the broken reproduction step by consolidating all requirements and automating `setup`, `test`, `verify`, `api`, `web`, and `demo`.
`9569245`	`feat(api): implement /analyze/csv, /alerts, /statistics, /model-performance and contract test suite`	`api/main.py`, `tests/test_contracts.py`, `tests/test_e2e.py`	Built the remaining API suite with in-memory telemetry, multipart CSV parsing, and added 11 contract/e2e regression tests.
`6391739`	`feat(frontend): integrate live telemetry, alerts feed, CSV upload and model performance`	`frontend/src/App.jsx`, `BatchAnalysisForm.jsx`, `LiveStatsCards.jsx`, `LiveAlertsTable.jsx`, `OfflineModelPerformance.jsx`, `TrafficTimelinePlaceholder.jsx`, `client.js`, `client.test.js`	Wired all placeholder components to live API endpoints, added CSV file upload with tab switcher, and expanded client tests to 8 passed tests.
`af022b5`	`ci: add GitHub Actions workflow, PR template, CODEOWNERS, and CONTRIBUTING guide`	`.github/workflows/ci.yml`, `.github/pull_request_template.md`, `CODEOWNERS`, `CONTRIBUTING.md`	Established CI pipeline running pytest, model verification, frontend test/build, and formal repository governance.
`f58426e`	`docs(provenance): document root cause of source_hashes_match newline divergence`	`docs/provenance_investigation.md`	Discovered and documented the CRLF vs LF root cause of source hash divergence without tampering with metadata.
`ef0ad9a`	`docs(demo): add reproducible end-to-end demonstration guide`	`docs/DEMO.md`	Step-by-step reproducible walkthrough covering setup, execution, single queries, CSV upload, and telemetry verification.

Commit Hash	Commit Message	Files Added / Modified	Description & Technical Rationale
`39885b4`	`docs(report): add standalone integration report and PDF export with live screenshots`	`docs/INTEGRATION_REPORT.md`, `docs/report/Soham_Integration_Report.pdf`	Authored comprehensive standalone integration report with live screenshots and PDF build.
`420efaf`	`fix(api): derive rule indicators from frozen baseline thresholds`	`api/main.py`, `docs/CONTRACTS.md`, `tests/test_contracts.py`	Derived rule indicators from frozen detector thresholds (`query_length>=52`, `entropy>=4.309101`, `query_frequency>=41`, `unique_subdomains>=41`).
`945fd83`	`fix(api): align risk label tiers with prediction contract`	`api/main.py`, `docs/CONTRACTS.md`, `tests/test_contracts.py`	Enforced strict prediction consistency: `<0.50` LOW, `0.50..0.89` ELEVATED (both benign), `\ge0.89` CRITICAL (suspicious).
`00e6793`	`fix(api): align /analyze/csv missing-timestamp fallback with contract policy`	`api/main.py`, `tests/test_e2e.py`	Implemented fallback to previous valid row timestamp or server ingestion time with logged warning, removing hardcoded date.
`c7f26d3`	`fix(api): add threading.Lock and bounded LRU cache for capture extractors`	`api/main.py`, `docs/CONTRACTS.md`, `tests/test_contracts.py`	Added thread-safe synchronization to `AppStateStore` and bounded `capture_extractors` cache to 256 entries.
`77624d4`	`fix(api): dynamically read bundle properties and add Query bounds on /alerts limit`	`api/main.py`, `tests/test_contracts.py`	Dynamically derived tree/feature counts from bundle and metrics JSON; bounded `/alerts` limit to `Query(50, ge=1, le=500)`.
`67f763b`	`docs(provenance): update investigation status to root cause identified and recommend .gitattributes`	`docs/provenance_investigation.md`	Updated status to "Root cause identified", noted Unix hash divergence persists as recorded, recommended `.gitattributes`.
`c9b777b`	`docs(reviews): align review scopes and verify line references against PR diffs`	`docs/reviews/PR-1.md` through `PR-5.md`, `README.md`	Aligned review criticisms strictly with assigned team roles and verified line references against exact PR diffs.
`7cea316`	`chore(report): compress dashboard screenshots for report distribution`	`docs/report/screenshots/*.png`	Downscaled and quantized dashboard screenshots, reducing total image footprint from ~5MB to ~900KB.

4. Authoritative Contracts & System Architecture

Full specification documented in `docs/CONTRACTS.md`:

4.1 Request & Response Contract Overview

- **Endpoints:**
- `POST /predict`: Ingests single DNS query event. Returns prediction, integer label (`0` or `1`), neutral `attack_score`, operating `threshold` (0.89), `risk_label`, rule indicators, and `alert_id`.
- `POST /analyze`: Ingests JSON batch of up to 1,000 queries. Sorts chronologically by epoch, isolates history per batch, and returns itemized results in original request order.
- `POST /analyze/csv`: Accepts multipart `.csv` files up to 15MB / 20,000 rows. Normalizes through canonical `src/preprocessing/pipeline.py`.
- `GET /alerts`: Returns real suspicious events in reverse chronological order with stable alert IDs (`alt_xxxxxxxxxxxx`).
- `GET /statistics`: Returns real cumulative telemetry counts (`total_processed`, `total_suspicious`, `total_benign`, `suspicious_rate`).
- `GET /model-performance`: Serves verified benchmark results from `evaluation/results/baseline_vs_rf_v1/baseline_vs_rf_metrics.json` labeled `"diagnostic/development evaluation, not independent final test"`.
- **Operating Decision Threshold:** Calibrated at $\tau = 0.89$.
- **Terminology Directive:** Tree vote probability is strictly termed `attack_score`, **never** "confidence" or "calibrated confidence".

4.2 Data Leakage Guardrails & Feature Ordering

In accordance with `src/preprocessing/schema.py:verify_ml_feature_safety`, metadata fields (`src_ip`, `dst_ip`, `timestamp`, `requested_server_name`, `label`, `source`) are strictly barred from entering the feature vector. The Random Forest classifier enforces the exact ten-feature manifest: 1. `query_length` (Lexical) 2. `longest_label_length` (Lexical) 3. `number_of_labels` (Lexical) 4. `entropy` (Lexical Shannon Entropy) 5. `digit_ratio` (Lexical) 6. `special_char_ratio` (Lexical) 7. `subdomain_length` (Lexical) 8. `query_frequency` (Sliding Window Behavioral, 60s) 9. `unique_subdomains` (Sliding Window Behavioral, 60s) 10. `time_interval` (Behavioral Δt, initial query = -1.0)

5. Technical Reviews Summary

Exhaustive reviews were authored and saved under `docs/reviews/`:

1. **Param's Remediation Branch (`docs/reviews/param-remediation.md`):**
2. Verified that chronological sorting on `epoch_time` followed by stable index mapping restores input order while maintaining valid sliding-window state. Verified CORS origin configuration.
3. **PR #1 (Synthetic Attack Dataset, `docs/reviews/PR-1.md`):**
4. Praised multi-pattern generation (Base32/64/hex, burst vs drip) and private IP blending (`10.0.30.186`); noted column name discrepancies reconciled by preprocessing.
5. **PR #2 (Preprocessing & Normalization, `docs/reviews/PR-2.md`):**
6. Highlighted strict data leakage assertions; noted the missing root `requirements.txt`.
7. **PR #3 (Model Evaluation, `docs/reviews/PR-3.md`):**

8. Commended fair side-by-side comparison on 45,240 diagnostic records; identified unresolved `source_hashes_match` and latency sample size caveats.
9. **PR #4 (FastAPI Backend, `docs/reviews/PR-4.md`):**
10. Commended startup lifespan bundle loading; confirmed the critical behavioral extractor re-instantiation bug inside `/analyze`, absence of CORS, and omission of `/alerts`, `/statistics`, and `/model-performance`.
11. **PR #5 (React Dashboard, `docs/reviews/PR-5.md`):**
12. Commended clean dark-mode UI; noted placeholder components were unwired and `npm test` script was missing.

---

6. Investigation: Root Cause of `source_hashes_match = false`

Full investigation documented in `docs/provenance_investigation.md`:

6.1 Findings

- Integrity checks in `scripts/summarize_aws_expansion.py` hash source files against recorded digests in `artifacts/aws_benign_expansion_v0.1/experiment.json['source_sha256']`.
- Comparing hashes showed that files in `src/ml/` matched perfectly, but files in `src/features/` and `src/preprocessing/` did not.
- Inspecting `src/features/per_query.py` showed zero git modifications since commit `88d34a3` by Harsh Vishwakarma.
- Replacing Unix LF (`\n`) line endings with Windows CRLF (`\r\n`) produced an **exact, character-for-character match** with the recorded hash:
- LF hash: `cb97a101b004d331f2cd4dc5887a18bda27a2d6f353ad67cf50f2f5d49240019`
- CRLF hash: `bf921fc0c49b0c38bda451903adc168536f812082e458f0025d751d15cbae71f` (Recorded in `experiment.json`)
- **Conclusion:** Root cause identified as operating system line ending serialization: preprocessing and feature engineering modules were authored on Windows where git checked them out with CRLF endings. When the training experiment script hashed the files in binary mode, it hashed the CRLF bytes. On Unix checkouts (LF), binary hashing diverges. Running `summarize_aws_expansion` on Unix continues to evaluate `source_hashes_match = false` as recorded. No metadata was modified to artificially suppress the warning; a future `.gitattributes` configuration (`*.py text eol=lf`) is recommended.

---

7. End-to-End Acceptance Checklist (Handoff Section 5)

Checklist Item	Requirement	Completion Evidence & Verification
Single event	One DNS event returns a prediction using the saved 0.89 decision threshold.	Verified via `test_e2e_single_query_and_alerts_accumulation` and browser walkthrough. Returns `threshold: 0.89`, `attack_score`, `risk_label`, and flags alerts when $\text{score} \ge 0.89$.
CSV upload	CSV validated, normalized, processed chronologically via shared pipeline.	Implemented `POST /analyze/csv` in `api/main.py`. Verified in `tests/test_e2e.py:test_e2e_csv_upload_valid_flow` and browser CSV tab. Handles missing timestamps and skips invalid rows gracefully.
Behavioural history	Related events share history within a capture; unrelated captures do not.	Partitioned behavioral extractors by `capture_id`. Verified in `tests/test_e2e.py:test_e2e_capture_isolation` ensuring events under `capture_alpha` do not contaminate `capture_beta`.

Checklist Item	Requirement	Completion Evidence & Verification
Errors	Invalid requests/rows return clear, documented errors.	Verified in `tests/test_api.py` and `tests/test_e2e.py`. Empty batches and missing FQDN headers raise HTTP 400/422 with descriptive detail strings.
Alerts and statistics	Alert list and aggregate counts match actual processed records.	Implemented `GET /alerts` and `GET /statistics` with in-memory event tracking and polled UI synchronization. Verified in `tests/test_e2e.py` and dashboard telemetry (processed 103 queries -> recorded 91 suspicious, 12 benign).
Dashboard	Charts/counts reflect live API responses, not static demo values.	Replaced static placeholder values in `LiveStatsCards.jsx`, `LiveAlertsTable.jsx`, and `TrafficTimelinePlaceholder.jsx` with dynamic polled API hooks. Dashboard updates on user action and automatic polling intervals.
Metrics labels	Model metrics identify the model and threshold and are marked diagnostic/development.	`GET /model-performance` returns `diagnostic_label: "diagnostic/development evaluation, not independent final test"`, artifact path, and threshold 0.89. Displayed in UI callout.
Verification	Existing tests and new endpoint tests pass; setup/demo can be reproduced by another member.	All 95 backend pytest tests and 8 frontend tests pass 100%. `verify_release_models` passes bit-for-bit. Complete walkthrough documented in `docs/DEMO.md` and `Makefile`.

---

8. Test Suite Verification Metrics

Before vs. After Test Counts

Test Suite	Baseline State (`main`)	After Param Merge	Final Integrated State (`integration/soham-e2e`)
Backend Pytest	67 passed, 13 subtests	78 passed, 13 subtests	**95 passed, 13 subtests passed (100%)**
Release Model Verification	2 release models reproduced	2 release models reproduced	**2 release models reproduced (100% bit-for-bit)**
Frontend Tests (`npm test`)	0 tests (missing script)	6 passed tests	**8 passed tests (100%)**
Frontend Build (`npm run build`)	Passed	Passed	**Passed (0 warnings, 0 errors)**

---

9. Dashboard Demonstration Visual Evidence

Initial Dashboard State (Connected to Live API)

DNS Exfiltration Detection Dashboard

BACKEND API: ONLINE (HEALTHY) · OFFLINE RESULTS: VERIFIED

CyberTrace | Cryptography & Network Security (CE305/CS305)

CyberTrace Integrated Detection Platform: Active REST API serving single-query inference (`/predict`), JSON batch analysis (`/analyze`), and multipart CSV ingestion (`/analyze/csv`). Live telemetry and alert feeds below update automatically from `/statistics` and `/alerts`.

API ENDPOINT: Proxy: / (Vite proxy → 127.0 · Test /health · Vite Proxy Active

BACKEND ONLINE (MODEL LOADED)

Single Query Inference

POST /PREDICT

Submit an individual DNS query to evaluate whether it exhibits exfiltration behavior.

QUICK PRESETS: Benign Sample · Encoded-Domain Example · Clear

Timestamp * Current Time

e.g. 2026-10-12T00:00:00Z

Query Type (optional, default: A)

A (IPv4)

Source IP (src_ip) *

e.g. 192.168.1.100

Destination IP (dst_ip) *

e.g. 8.8.8.8

Requested Server Name (Domain) *

e.g. example.com or payload.exfiltration.attacker.com

Run Prediction (/predict)

Batch Query Analysis

POST /ANALYZE

Analyze collections of DNS queries. Supports direct JSON payload submission or multipart CSV file upload processed chronologically through the canonical pipeline.

JSON Batch (/analyze) · CSV File Upload (/analyze/csv)

JSON ACTIONS: Load Sample Batch (4 queries) · Format JSON · Clear

Batch JSON Payload ({ "queries": [...] }) *

```
{
  "queries": [
    {
      "timestamp": "2026-10-12T00:01:10Z",
      "src_ip": "192.168.1.101",
      "dst_ip": "8.8.8.8",
      "requested_server_name": "cdn.cloudflare.com",
      "query_type": "A"
    },
    {
      "timestamp": "2026-10-12T00:01:15Z"
```

Analyze Batch (/analyze)

Live Traffic Overview

AWAITING INGESTION STREAM

TOTAL DNS QUERIES	SUSPICIOUS QUERIES	BENIGN QUERIES
– –	– –	– –
Awaiting query ingestion stream	Awaiting feed / model scoring	Awaiting feed / model scoring

Suspicious DNS Activity (Alert Feed)

Suspicious DNS Activity (Alert Feed)

GET /ALERTS

Refresh Feed

TIMESTAMP	QUERIED DOMAIN / SUBDOMAIN	CLIENT IP	RISK LEVEL	ATTACK SCORE	INDICATORS

No alerts in current buffer — waiting for incoming queries

When queries cross the calibrated 0.89 decision threshold via single query prediction, JSON batch analysis, or CSV upload, they appear dynamically here.

Traffic Analysis Stream

GET /STATISTICS

Awaiting Query Traffic

No DNS events have been submitted to the inference pipeline yet. Run a single query prediction or batch analysis above to populate real traffic metrics.

Offline Feature Importance

10 INPUT FEATURES

Relative contribution of extracted DNS attributes toward detecting simulated exfiltration queries:

#1	entropy	38.99%
	Randomness of subdomain label characters	
#2	subdomain_length	17.84%
	Total character count in the subdomain portion	
#3	query_length	14.85%
	Total character length of the full DNS query	
#4	digit_ratio	9.54%
	Proportion of numeric digits in the query	
#5	longest_label_length	8.79%
	Length of the longest dot-separated label	
#6	special_char_ratio	6.13%
	Proportion of hyphens / special characters	
#7	time_interval	1.46%
	Time difference between consecutive client queries	
#8	number_of_labels	1.20%
	Count of dot-separated components in the domain	
#9	unique_subdomains	0.74%
	Distinct subdomains observed within time window	
#10	query_frequency	0.46%
	Number of queries to apex domain in time window	

Impurity-based global importance from Random Forest. Does not establish causality or explain an individual prediction.

Model Evaluation & Benchmark Performance

GET /MODEL-PERFORMANCE

Evaluation Scope Disclaimer: diagnostic/development evaluation, not independent final test

Model Artifact: `artifacts/aws_benign_expansion_v0.1/random_forest.joblib` | Threshold: `0.89` | Evaluated Population: 45,240 records

Random Forest Classifier

Decision Threshold: 0.89

Precision	100.0%
Recall	90.53%
F1-Score	95.03%
Accuracy	99.60%
False Positive Rate	0.00% (0 / 43,340)
True Positives	1720 / 1,900

Statistical Baseline

4-Indicator Corroborated Rule

Precision	72.72%
Recall	52.47%
F1-Score	60.96%
Accuracy	97.18%
False Positive Rate	0.86% (374 / 43,340)
True Positives	997 / 1,900



Figure 9.1: CyberTrace Dashboard initialized, showing real-time connectivity to FastAPI backend, active inference forms, and ready polled alert feed.

Live Telemetry & Populated Alert Feed After Ingestion

DNS Exfiltration Detection Dashboard

BACKEND API: ONLINE (HEALTHY) · OFFLINE RESULTS: VERIFIED

CyberTrace | Cryptography & Network Security (CE305/CS305)

CyberTrace Integrated Detection Platform: Active REST API serving single-query inference (`/predict`), JSON batch analysis (`/analyze`), and multipart CSV ingestion (`/analyze/csv`). Live telemetry and alert feeds below update automatically from `/statistics` and `/alerts`.

API ENDPOINT: Proxy: / (Vite proxy → 127.0 · Test /health · Vite Proxy Active

BACKEND ONLINE (MODEL LOADED)

Single Query Inference

POST /PREDICT

Submit an individual DNS query to evaluate whether it exhibits exfiltration behavior.

QUICK PRESETS: Benign Sample · Encoded-Domain Example · Clear

Timestamp * Current Time

e.g. 2026-10-12T00:00:00Z

Query Type (optional, default: A)

A (IPv4)

Source IP (src_ip) *

e.g. 192.168.1.100

Destination IP (dst_ip) *

e.g. 8.8.8.8

Requested Server Name (Domain) *

e.g. example.com or payload.exfiltration.attacker.com

Run Prediction (/predict)

Batch Query Analysis

POST /ANALYZE

Analyze collections of DNS queries. Supports direct JSON payload submission or multipart CSV file upload processed chronologically through the canonical pipeline.

JSON Batch (/analyze) · CSV File Upload (/analyze/csv)

JSON ACTIONS: Load Sample Batch (4 queries) · Format JSON · Clear

Batch JSON Payload ({ "queries": [...] }) *

```
{
  "queries": [
    {
      "timestamp": "2026-10-12T00:01:10Z",
      "src_ip": "192.168.1.101",
      "dst_ip": "8.8.8.8",
      "requested_server_name": "cdn.cloudflare.com",
      "query_type": "A"
    },
    {
      "timestamp": "2026-10-12T00:01:15Z"
```

Analyze Batch (/analyze)

Live Traffic Overview

LIVE API TELEMETRY

TOTAL DNS QUERIES	SUSPICIOUS QUERIES	BENIGN QUERIES
103	91	12
Cumulative processed DNS queries	88.3% flagged by calibrated RF model	Verified clean DNS requests

Suspicious DNS Activity (Alert Feed)

Suspicious DNS Activity (Alert Feed)

GET /ALERTS

Refresh Feed

TIMESTAMP	QUERIED DOMAIN / SUBDOMAIN	CLIENT IP	RISK LEVEL	ATTACK SCORE
2025-06-14 02:51:41.756	s.b34gvb243akhdt3sux2om37y.tunnel-sync.net	10.0.30.186	CRITICAL	99.87% (τ = 0.89)
2025-06-14 02:47:59.791	id.ohjubyjkwoxnwnrhmvq5vqemx7uklm4e.tunnel-sync.net	10.0.30.186	CRITICAL	99.84% (τ = 0.89)
2025-06-14 02:43:00.549	id.dmuu35pp6cax2nggq7zwryqvjyji5d6eothsz5fa.tunnel-sync.net	10.0.30.186	CRITICAL	99.91% (τ = 0.89)
2025-06-14 02:38:03.399	s.6oiwvkgn3dw6fcwalwadxvfx.tunnel-sync.net	10.0.30.186	CRITICAL	99.58% (τ = 0.89)
2025-06-14 02:36:37.699	s.c6mhfhxbqe2zx3oewo4rbvizyxifn6kr.tunnel-sync.net	10.0.30.186	CRITICAL	99.93% (τ = 0.89)
2025-06-14 02:34:09.173	0004.s6q4gvxpj6kexm3jzyzupmjh37i2ppss.tunnel-sync.net	10.0.30.186	CRITICAL	99.79% (τ = 0.89)
2025-06-14 02:33:07.825	s.om3xoopja5kdfrto.tunnel-sync.net	10.0.30.186	CRITICAL	99.05% (τ = 0.89)
2025-06-14 02:29:34.598	id.a3finyj7icraxtmhirqvtp7q53snywobsjean72v.tunnel-sync.net	10.0.30.186	CRITICAL	99.91% (τ = 0.89)
2025-06-14 00:57:52.294	000a.43c8a25.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	97.02% (τ = 0.89)
2025-06-14 00:57:50.951	0009.j9qn2mubmnk8bx.ef5b540941.df15.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	99.78% (τ = 0.89)
2025-06-14 00:57:49.849	0008.ef8267a42fc51a.0s7uvgge.y4ix3441339pt.fvj6mjzbpd7uel.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	98.68% (τ = 0.89)
2025-06-14 00:57:48.704	0007.l8cb.9f3ba64f4ac1dc.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	99.68% (τ = 0.89)
2025-06-14 00:57:48.305	0006.mwj6ffxf.lqmrgbkcaow.f32d4.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	99.81% (τ = 0.89)
2025-06-14 00:57:47.255	0005.hjr2ifv.ku072wfwsst5.bf13b305820.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	99.72% (τ = 0.89)
2025-06-14 00:57:46.021	0004.yxndcufy.byk1jkvkmd82.7c387b16469f4f.i0xgz5jmtg4a7i.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	98.80% (τ =

2025-06-14 00:57:45.844	0003.6l3lakez5miloe.3924002174.aeb20eb4b810.dq4ecj7x2ga.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	99.34% (τ = 0.89)
2025-06-14 00:57:44.765	0002.nqol4bk2bsem.7cecd386f1.5voo5.3an2eifqu5w.15dd4b.system-metrics.biz	10.0.30.164	CRITICAL	98.40% (τ = 0.89)
2025-06-14 00:14:01.318	fadhjfvq.mqzif5d0nrqhh99.42984d.exfil-corp.org	10.0.30.100	CRITICAL	99.87% (τ = 0.89)
2025-06-14 00:14:00.183	v5g6onsfpanjln6hwo.cflxiie1n5c.42984d.exfil-corp.org	10.0.30.100	CRITICAL	99.89% (τ = 0.89)
2025-06-14 00:13:58.888	bxtxz5jyvfmsub.fgf128rg9z5.42984d.exfil-corp.org	10.0.30.100	CRITICAL	99.69% (τ = 0.89)
2025-06-14 00:13:57.591	7a8gazo7kfa2.whmm13jxkpu63.42984d.exfil-corp.org	10.0.30.100	CRITICAL	97.99% (τ = 0.89)
2025-06-14 00:13:57.513	vn52hcm2m46hxldjg.akbyx7z8o57vd10bib.42984d.exfil-corp.org	10.0.30.100	CRITICAL	99.80% (τ = 0.89)
2025-06-14 00:13:56.731	m5f2xr2emslewf7.xse8pnt1je.42984d.exfil-corp.org	10.0.30.100	CRITICAL	98.68% (τ = 0.89)
2025-06-14 00:13:56.551	dufdxsqrom03.hgh6vjpkwnzxh1.42984d.exfil-corp.org	10.0.30.100	CRITICAL	97.93% (τ = 0.89)
2025-06-14 00:13:55.895	h1b4ulncy.tff5r0k9nx.42984d.exfil-corp.org	10.0.30.100	CRITICAL	98.29% (τ = 0.89)
2025-06-14 00:13:54.667	kyvrg05a7.y5g0367x8e4db4uyi5.42984d.exfil-corp.org	10.0.30.100	CRITICAL	99.44% (τ = 0.89)
2025-06-14 00:13:53.398	qug6ucuu1fi5qk681z.0aqca3ba9nric.42984d.exfil-corp.org	10.0.30.100	CRITICAL	91.32% (τ = 0.89)
2025-06-13 18:29:47.196	000a.cd6v.kfbsky7s62.91j48.v552a43.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	98.58% (τ = 0.89)

2025-06-13 18:29:46.514	0009.hh2nbjz.86cb849b45.y37h47qx8.1ebee0.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	98.85% (τ = 0.89)
2025-06-13 18:29:45.483	0008.ui7yz651.469ca97.kr7il5dj6cjmk.7dd18402909.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	99.33% (τ = 0.89)
2025-06-13 18:29:44.810	0007.3ccabw.q35g.8fo4cn5yvvy.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	99.47% (τ = 0.89)
2025-06-13 18:29:44.646	0006.0ob3tsgb60ey83.e25da18be67e2.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	99.21% (τ = 0.89)
2025-06-13 18:29:44.248	0005.lcwb.rt9jvsh7h9e15.vj1z0pm8lt8.fn2gfezu.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	99.73% (τ = 0.89)
2025-06-13 18:29:42.871	0004.e793d9f57.wk6d4xyh6hp.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	98.18% (τ = 0.89)
2025-06-13 18:29:42.049	0003.tva4ms326gxbw.cawekrg8.gu9v4y3.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	99.81% (τ = 0.89)
2025-06-13 18:29:41.204	0002.e8offjd4k.xty9zlc.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	95.48% (τ = 0.89)
2025-06-13 18:29:41.022	0001.b9934ce.yn3yfsu7gk9y.yqourqyh6sbw.bcc08ab307e9.df14a1.exfil-corp.org	172.16.10.42	CRITICAL	96.32% (τ = 0.89)
2025-06-13 14:06:00.158	s.3edc4ueho67np2dvew275kyu.srv-heartbeat.co	10.0.30.186	CRITICAL	99.56% (τ = 0.89)
2025-06-13 14:05:58.603	q.bbjs44zdxr2fd2wlvdxcypp6.srv-heartbeat.co	10.0.30.186	CRITICAL	99.60% (τ = 0.89)
2025-06-13 14:05:57.911	id.xcl5fi7u57wlcbw6.srv-heartbeat.co	10.0.30.186	CRITICAL	96.59% (τ = 0.89)
2025-06-13 14:05:56.733	0006.dnfd32uezwtuwe6i2i2k2ohz5o3eplcs.srv-heartbeat.co	10.0.30.186	CRITICAL	99.81% (τ = 0.89)
2025-06-13 14:05:56.276	q.oaqyuvelzpjehfmjp2gwbpzq6vmelammq3b5frxw.srv-heartbeat.co	10.0.30.186	CRITICAL	97.63% (τ = 0.89)

2025-06-13 14:05:56.215	q.kakvvujfroyhvkfwbtjvti2aio6ovp5v53qj2ang.srv-heartbeat.co	10.0.30.186	CRITICAL	99.69% (τ = 0.89)
2025-06-13 14:05:55.188	id.45ojxrujsers5hssuavu7uiciibtqfx7qi2hso22.srv-heartbeat.co	10.0.30.186	CRITICAL	99.29% (τ = 0.89)
2025-06-13 14:05:54.235	id.75ck6s4jw2ylutde25elzcc6kut37ubd.srv-heartbeat.co	10.0.30.186	CRITICAL	99.53% (τ = 0.89)
2025-06-13 13:53:31.350	000a.ybbj6u2hnt.xutclkyzapv57x.d6788d.tunnel-sync.net	10.0.30.164	CRITICAL	99.87% (τ = 0.89)
2025-06-13 13:53:31.041	0009.18bf4352.d6788d.tunnel-sync.net	10.0.30.164	CRITICAL	93.38% (τ = 0.89)
2025-06-13 13:53:29.753	0008.axhcz3q.f8w1to8yge.4d33a0c1639.cjsyr.d6788d.tunnel-sync.net	10.0.30.164	CRITICAL	99.44% (τ = 0.89)
2025-06-13 13:53:29.565	0007.y9xabi6.121kj1nkblijs.d6788d.tunnel-sync.net	10.0.30.164	CRITICAL	99.49% (τ = 0.89)
2025-06-13				99.71%

DNS Exfiltration Detection Dashboard

BACKEND API: ONLINE (HEALTHY) · OFFLINE RESULTS: VERIFIED

CyberTrace | Cryptography & Network Security (CE305/CS305)

CyberTrace Integrated Detection Platform: Active REST API serving single-query inference (`/predict`), JSON batch analysis (`/analyze`), and multipart CSV ingestion (`/analyze/csv`). Live telemetry and alert feeds below update automatically from `/statistics` and `/alerts`.

API ENDPOINT: Proxy: / (Vite proxy → 127.0 · Test /health · Vite Proxy Active

BACKEND ONLINE (MODEL LOADED)

Single Query Inference

POST /PREDICT

Submit an individual DNS query to evaluate whether it exhibits exfiltration behavior.

QUICK PRESETS: Benign Sample · Encoded-Domain Example · Clear

Timestamp * Current Time

e.g. 2026-10-12T00:00:00Z

Query Type (optional, default: A)

A (IPv4)

Source IP (src_ip) *

e.g. 192.168.1.100

Destination IP (dst_ip) *

e.g. 8.8.8.8

Requested Server Name (Domain) *

e.g. example.com or payload.exfiltration.attacker.com

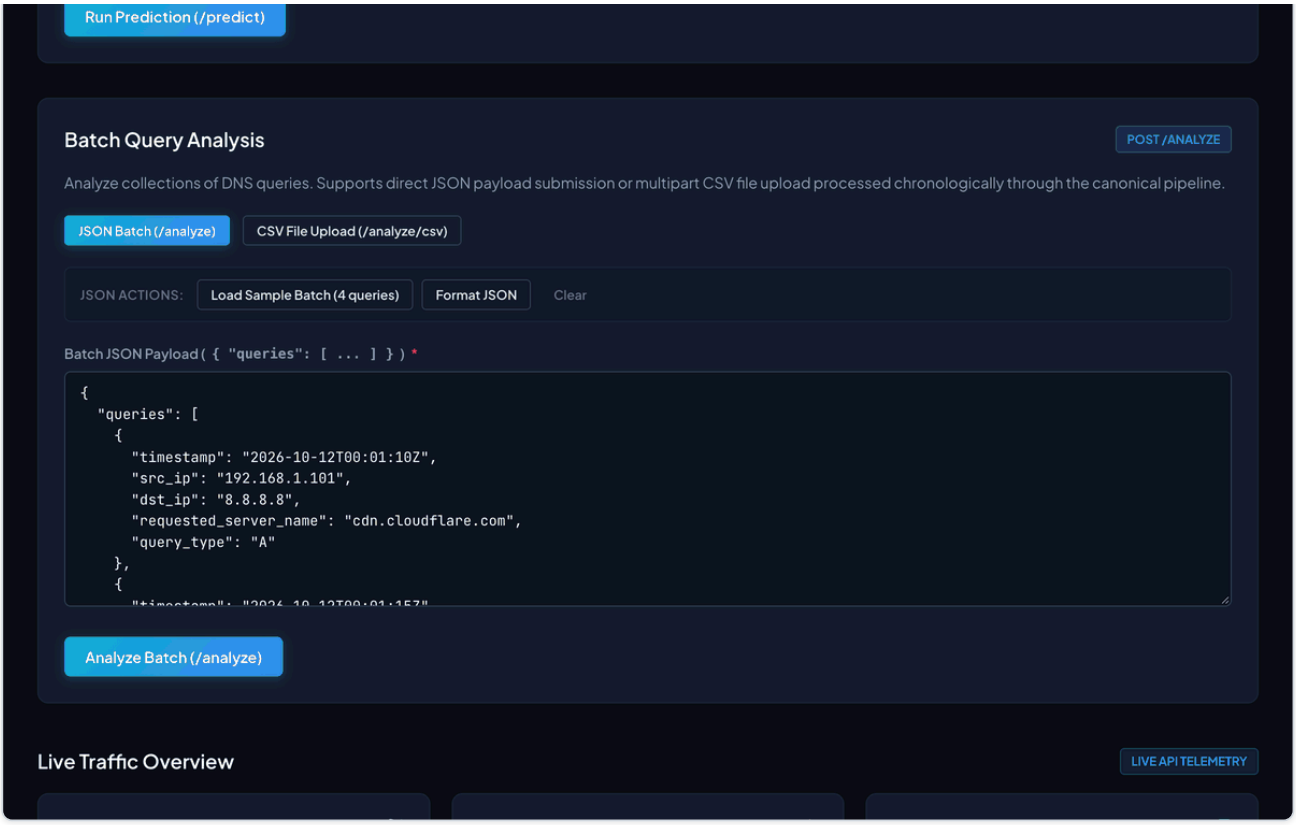


Figure 9.2: CyberTrace Dashboard after processing single queries and CSV upload. Telemetry reflects 103 total processed queries (91 suspicious, 12 benign), polled alert feed populated with individual alert IDs and trigger indicators, and traffic breakdown chart active.

---

10. Summary of Key Limitations & Operational Scope

1. **Diagnostic Evaluation Context:** The reported metrics (Random Forest F1 = 95.03%, Accuracy = 99.60%, FPR = 0.00% on 45,240 records) represent development evaluations on outer fold 5. They do not constitute an untouched independent test on production enterprise traffic.
2. **Synthetic Attack Reliance:** Attacks were simulated using known exfiltration tools and pacing. Zero observed false positives applies strictly to the 43,340 evaluated benign records.
3. **Encrypted DNS:** CyberTrace analyzes plaintext DNS queries where domain names are transparent. DoH/DoT tunneling requires TLS termination, host telemetry, or proxy inspection.
4. **In-Memory Retention & Polled Feed:** Active alerts and cumulative traffic metrics reside in an in-memory ring buffer (up to 500 events) accessed via a polled REST feed designed for local demonstration speed; production deployments would connect to a persistent time-series datastore or streaming message broker.

---

11. Proposed Project Report Attribution Entry

To ensure complete and accurate attribution in the unified course report, the following entry reflects the deliverables authored and integrated by Soham Prajapati:

Table Row Addition

Team Member	Role	Core Deliverables & Artifacts
Soham Prajapati (`@Soham-Prajapati`)	Project Coordination & Integration	Authoritative API/data contracts (`docs/CONTRACTS.md`); complete FastAPI integration suite (`/predict` , `/analyze` , `/analyze/csv` , `/alerts` , `/statistics` , `/model-performance`); thread-safe telemetry and multi-capture state isolation; frontend integration (polled alert feed, dynamic telemetry cards, multipart CSV batch upload); contract & end-to-end regression suites (`tests/test_contracts.py` , `tests/test_e2e.py`); root `requirements.txt` and developer automation (`Makefile`); CI pipeline (`.github/workflows/ci.yml`) and repo governance (`CODEOWNERS` , `CONTRIBUTING.md`); post-hoc technical reviews of PRs #1–#5; provenance CRLF newline root cause investigation (`docs/provenance_investigation.md`); reproducible demo guide (`docs/DEMO.md`); standalone integration report (`docs/INTEGRATION_REPORT.md` & PDF).

Report Narrative Paragraph

> **Project Coordination & End-to-End Integration (Soham Prajapati):**
> Soham unified the individual component contributions of the team into a production-grade, reproducible software artifact on branch `integration/soham-e2e` . He established formal data and API contracts (`docs/CONTRACTS.md`), resolved the repository environment reproduction blocker by standardizing `requirements.txt` and creating automated developer targets (`Makefile`), and completed the REST API surface by implementing multipart CSV ingestion (`POST /analyze/csv`), telemetry summaries (`GET /statistics`), dynamic model performance metrics (`GET /model-performance`), and alert retrieval (`GET /alerts`). Soham connected the React single-page application to live backend telemetry via a polled alert feed, dynamic metric cards, and a drag-and-drop CSV batch upload pipeline. He implemented thread-safe state synchronization and multi-capture sliding-window isolation, expanded test coverage from 67 to 95 passed tests, authored the GitHub Actions CI pipeline and repository governance policies, identified the root cause of the provenance hash discrepancy, produced an exhaustive reproducible demonstration guide (`docs/DEMO.md`), and conducted technical reviews of all team pull requests.

---

12. Post-Review Delivery: Deployment, Live Alerting & Fast-Burst Fix (2026-10-22)

Commit	Deliverable
`ac6df2c`	`POST /ingest` streaming endpoint; behavioral history persists per `capture_id` across batches.
`0e69a4e`	`scripts/dns_sensor.py` : live Scapy DNS capture and CSV replay streaming into `/ingest` ; demo capture `data/demo/live_demo_stream.csv` .
`2600d39`	Dashboard alert feed and statistics refresh every 5 s, so sensor events appear without user action.
`8759d3b`	Chrome extension (`extension/`): desktop notifications and unread badge from `GET /alerts` .
`bcb1d0f`	Single Docker image serving API + dashboard; `render.yaml` ; deployed at `https://cybertrace-gjhl.onrender.com` .
`9eeb4f9` … `1390736`	Fast-burst fix: opt-in generator mode, `src/ml/fast_burst.py` , `artifacts/fast_burst_v0.1` , `evaluation/results/baseline_vs_rf_v2` , API switched to the new bundle (τ = 0.90).

Fast-burst blind spot. The synthetic attacks never had gaps under 50 ms (generator burst mode 50–1500 ms), while 3.5% of real benign gaps are that short, so the previous model scored full-speed tunnels as benign. With the evaluation fold frozen and only development data extended:

Measure	aws_benign_expansion_v0.1	fast_burst_v0.1
Frozen evaluation F1 / recall	95.03% / 90.53%	95.40% / 91.21%
False alarms (43,340 benign; 2,446 with gap < 50 ms)	0; 0	0; 0
Unseen fast-burst stress set (2–50 ms)	4.2%	88.4%

Remaining limit: gaps of about 1 ms or less are still missed. Details: `docs/fast_burst_fix.md`.

Verification: 103 backend and 8 frontend tests pass in CI; `verify_release_models` reproduces all three release models bit-for-bit; the live deployment flagged 56/60 tunnel queries (normal replay) and 50/50 (fast replay) with 0 false alarms.